Lecture 6: Relational Design Theory

From Good Ideas to Great Databases

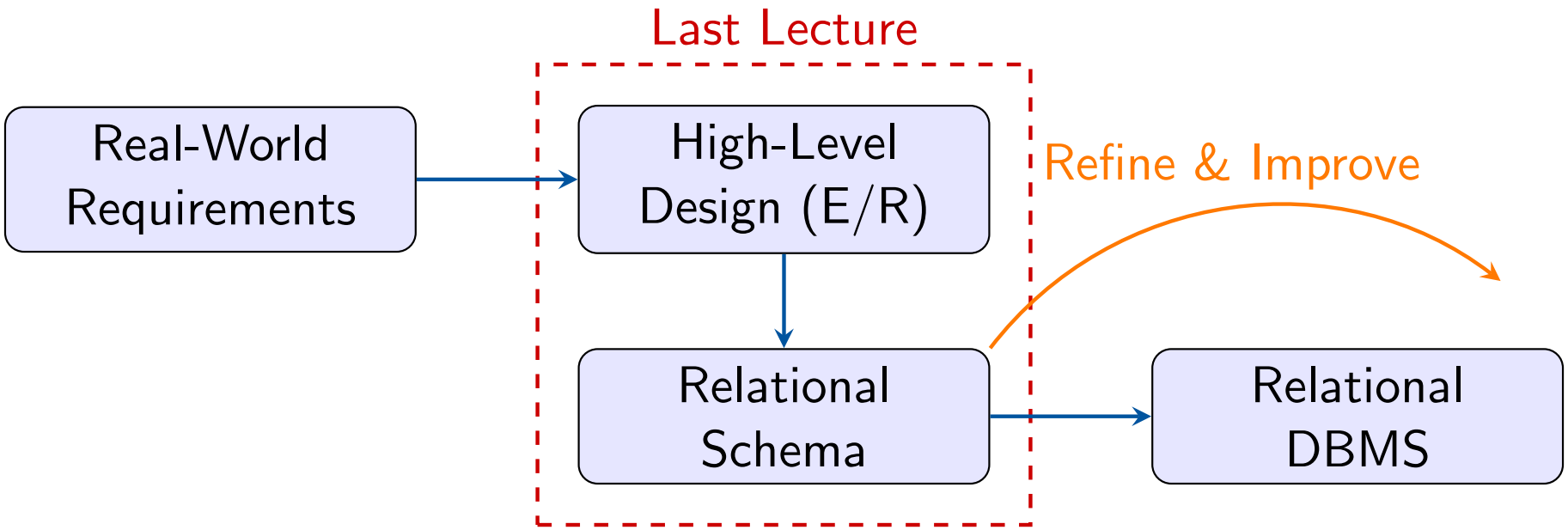
The Scenario

Imagine you're the lead architect for a new music streaming service. Your team's prototype stores all data in one massive table:

`TrackPlays(UserID, UserName, UserEmail, TrackID, TrackTitle, ArtistName, PlayTimestamp)`

This seems simple and works for the demo. But as you prepare to scale to millions of users, what are the hidden dangers lurking in this schema?

From Good Ideas to Great Databases



Goal

Learn the theory of **Functional Dependencies** to diagnose design flaws and discover the **Keys** that uniquely identify our data.

Normal Forms

- 1st Normal Form (1NF) = All tables are flat
- 2nd Normal Form = There are no partial dependencies of non-key attributes on part of a composite key.
- DB designs based on functional dependencies, intended to prevent data anomalies
  - Boyce-Codd Normal Form (BCNF)
  - 3rd Normal Form (3NF)
- 4th and 5th Normal Forms = see **Database Systems: The Complete Book (2nd edition)**
  - Chapter 3: Design Theory for Relational Databases (Sections 3.1 – 3.5)

Agenda

1. **The Problem**: Data Anomalies in "Bad" Designs
2. **The Tool**: Functional Dependencies (FDs)
3. **The Method**: Finding All Implied FDs with Closures
4. **The Goal**: Using FDs to Find Keys

1. The Problem: Data Anomalies

Starting Point: First Normal Form (1NF)

1NF Requirement

All attribute values must be **atomic**. This means no lists, sets, or complex objects within a single cell. Every table must be "flat".

Violates 1NF

Student	Courses
Mary	{CS145, CS229}
Joe	{CS145, CS106}

Not a valid relational table!

In 1st NF

Student	Course
Mary	CS145
Mary	CS229
Joe	CS145
Joe	CS106

This is a valid starting point.

Assumption

From now on, we assume all our relations are in 1NF.

The Story of a Bad Design: A Music App

Let's look at a few rows from our `TrackPlays` table.

TrackPlays(UserID, UserName, TrackID, TrackTitle)

UserID	UserName	TrackID	TrackTitle
U101	Alice	T505	"Starlight"
U101	Alice	T800	"Uprising"
U202	Bob	T505	"Starlight"

Problem 1: Redundancy
The user name 'Alice' is repeated for every song she plays. The track title "Starlight" is repeated for every user who plays it.

Data Anomalies: The Update Problem

Redundancy directly leads to update anomalies. What happens if Alice changes her user name to 'Alicia'?

TrackPlays(UserID, UserName, TrackID, TrackTitle)

UserID	UserName	TrackID	TrackTitle
U101	Alicia	T505	"Starlight"
U101	Alice	T800	"Uprising"
U202	Bob	T505	"Starlight"

Update Anomaly
If we forget to update her name in *every single row*, our data becomes inconsistent. What is user U101's name? The database gives conflicting answers.

Data Anomalies: The Deletion Problem

Our bad design can cause us to lose information unintentionally.

TrackPlays(UserID, UserName, TrackID, TrackTitle)

UserID	UserName	TrackID	TrackTitle
U303	Carol	T999	"Unreleased Demo"

Deletion Anomaly

If Carol is the only person who ever played this rare demo, and she closes her account, deleting this row makes us lose the fact that the track "Unreleased Demo" ever existed.

Data Anomalies: The Insertion Problem

Our bad design can also prevent us from storing valid information.

TrackPlays(UserID, UserName, TrackID, TrackTitle)

UserID	UserName	TrackID	TrackTitle
U101	Alice	T505	"Starlight"
U202	Bob	T505	"Starlight"
???	???	T123	"New Album Intro"

Insertion Anomaly

We can't add a new track to our catalog until at least one user plays it. The schema forces us to have a UserID, but we don't have one yet!

The Solution: Decomposing the Relation

We can solve all these anomalies by splitting the relation into smaller, well-structured ones.

Users

UserID	UserName
U101	Alice
U202	Bob

Tracks

TrackID	TrackTitle
T505	"Starlight"
T800	"Uprising"

Plays

UserID	TrackID
U101	T505
U101	T800
U202	T505

- **Redundancy?** Gone. User and track info is stored once.
- **Update Anomaly?** Solved. Change a user's name in one single place.
- **Deletion Anomaly?** Solved. Deleting a play doesn't delete the user or the track.
- **Insertion Anomaly?** Solved. We can add new users and tracks anytime.

The Big Question

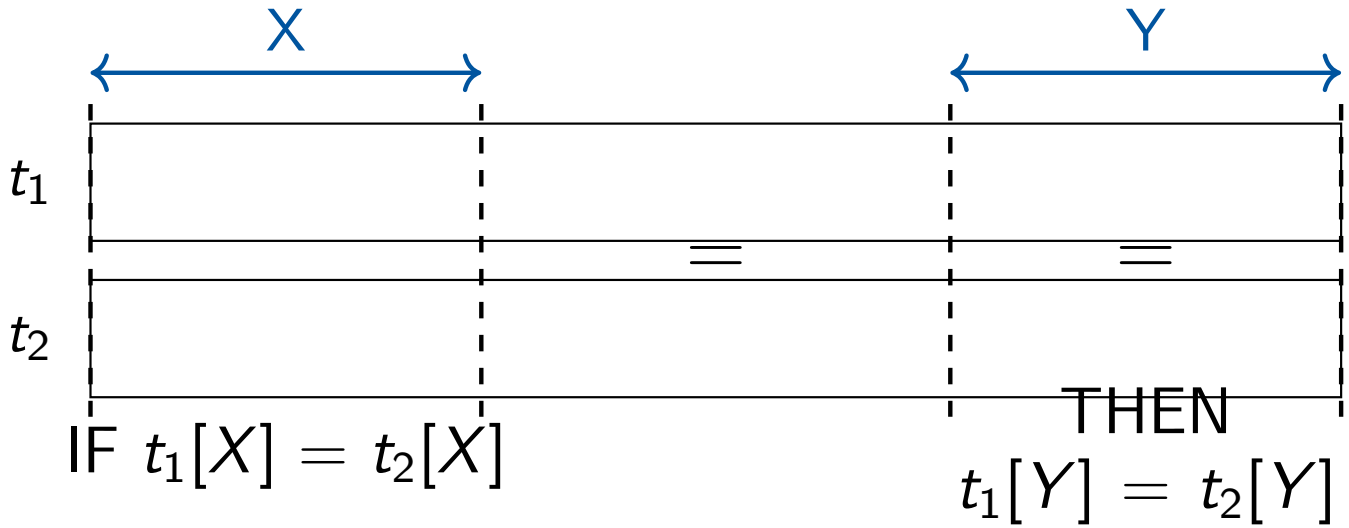
This seems intuitive, but how do we find the right decomposition systematically for any database?

2. The Tool: Functional Dependencies

Functional Dependency (FD)

Definition

A functional dependency $X \rightarrow Y$ is a **constraint** on a relation R. It states that for any two tuples t_1 and t_2 in R, if they have the same values for all attributes in set X, they **must** also have the same values for all attributes in set Y.



In Plain English

The values of the attributes in X **uniquely determine** the values of the attributes in Y. In our example: `UserID` $\rightarrow$ `UserName, UserEmail`.

Splitting/Combining Rule for FDs

Rule

You can split or combine attributes on the **RIGHT-HAND SIDE** of an FD.

$$A_1 \ldots A_n \rightarrow B_1 B_2 \ldots B_m$$

$$\Updownarrow \text{ is equivalent to}$$

$$A_1 \ldots A_n \rightarrow B_1, \quad A_1 \ldots A_n \rightarrow B_2, \quad \ldots, \quad A_1 \ldots A_n \rightarrow B_m$$

Example

`title, year` $\rightarrow$ `length, genre` is the same as the two FDs:

- `title, year` $\rightarrow$ `length`
- `title, year` $\rightarrow$ `genre`

A Common Misconception: The Left Side

Warning!

You **CANNOT** split attributes on the **LEFT-HAND SIDE** of an FD.

$$\texttt{title, year} \rightarrow \texttt{length} \not\Rightarrow$$ This does **NOT** imply:

- $\texttt{title} \rightarrow \texttt{length}$ (A movie and its remake can have the same title but different lengths).
- $\texttt{year} \rightarrow \texttt{length}$ (Many movies are released in the same year with different lengths).

FDs are Schema-Level Constraints

- An FD is a statement about what must be true for **all possible legal instances** of the database. It's a business rule we assert about the data.

This instance SATISFIES

`UserID` $\rightarrow$ `UserName`

UserID	UserName
U101	Alice
U101	Alice
U202	Bob

This instance VIOLATES

`UserID` $\rightarrow$ `UserName`

UserID	UserName
U101	Alice
U101	Alicia
U202	Bob

Analogy: The Law of Data Gravity

An FD is a **physical law** of your data's universe (e.g., 'Gravity exists'). A database instance is an **observation** (e.g., 'This apple fell'). You can use one observation of a floating apple to *disprove* the law of gravity. But you can never *prove* the law is universal just by watching a million apples fall.

Trivial Functional Dependencies

- An FD $X \rightarrow Y$ is **trivial** if the right-hand side is a subset of the left-hand side ($Y \subseteq X$).
- Trivial FDs are always true by definition.

Example of a Trivial FD

`UserID, UserName` $\rightarrow$ `UserName`

This is obviously true. If two rows have the same UserID and UserName, they must have the same UserName.

Why We Care (or Don't)

We call these 'trivial' because they don't tell us anything new about the relationships *between different* attributes. We can safely ignore them in our design process.

Exercise #1: Data Detective

For the data instance below, find:

1. A non-trivial FD that appears to hold.
2. A non-trivial FD that is clearly violated.

EmpID	**Name**	**Phone**	**Position**
E0045	Smith	1234	Clerk
E3542	Mike	9876	Salesrep
E1111	Smith	9876	Salesrep
E9999	Mary	1234	Lawyer

Exercise #1: Data Detective

For the data instance below, find:

1. A non-trivial FD that appears to hold.
2. A non-trivial FD that is clearly violated.

EmpID	Name	Phone	Position
E0045	Smith	1234	Clerk
E3542	Mike	9876	Salesrep
E1111	Smith	9876	Salesrep
E9999	Mary	1234	Lawyer

Solution

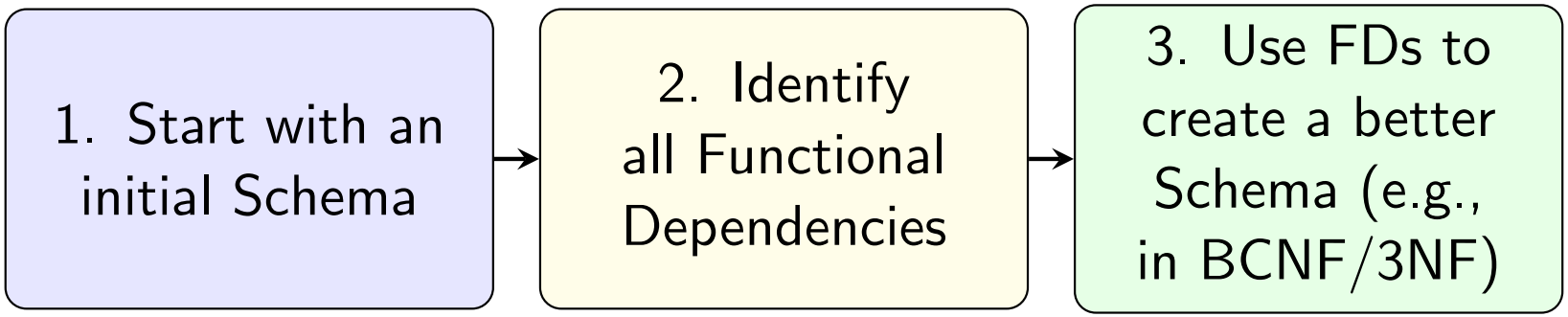
1. **Holds**: `EmpID` $\rightarrow$ `Name, Phone, Position`. (Every EmpID is unique, so it determines everything else in this instance). Also `Position` $\rightarrow$ `Phone` appears to hold.
2. **Violated**: `Name` $\rightarrow$ `EmpID` (Two rows have 'Smith' but different EmpIDs). Also `Phone` $\rightarrow$ `Position` ('1234' maps to both Clerk and Lawyer).

3. The Method: Finding Implied FDs

The Design Process with FDs

Our Goal

FDs are the key to unlocking good database design.



The Challenge

Sometimes FDs imply other FDs. We need a systematic way to find **all** implied FDs.

The Logic Engine: Armstrong's Axioms

Given a set of FDs, we can derive all implied FDs using three self-evident rules:

1. **Reflexivity:** If you know a person's `(Name, Address)`, you certainly know their `Name`.
   - Formally: If $Y \subseteq X$, then $X \to Y$.
2. **Augmentation:** If `StudentID` $\to$ `Major`, then it's also true that `(StudentID, Year)` $\to$ `(Major, Year)`.
   - Formally: If $X \to Y$, then $XZ \to YZ$.
3. **Transitivity:** If `EmpID` $\to$ `DeptID`, and `DeptID` $\to$ `Manager`, then `EmpID` $\to$ `Manager`.
   - Formally: If $X \to Y$ and $Y \to Z$, then $X \to Z$.

The Takeaway

Using these axioms directly is tedious. Instead, we use an algorithm that systematically applies this logic for us. The axioms are the guarantee that the algorithm is logically sound.

A Better Way: Attribute Closure

The Big Idea

The **closure** of a set of attributes X (written X^+) is the set of all attributes that are functionally determined by X.

Analogy: The Expanding Snowball

The closure algorithm works like a snowball rolling downhill.

- You start with a small core of attributes ('X'). On each pass, you scan your FDs. If your snowball is big enough to contain the left side of an FD, it 'picks up' the attributes on the right side, growing larger.
- You keep rolling until a full pass adds no new snow.
- The final snowball is the closure.

A Better Way: Attribute Closure

The Closure Algorithm

To compute X^+ given a set of FDs, F:

1. Initialize 'closure = X'.
2. Repeatedly scan through all FDs in F. For each FD $Y \rightarrow Z$:
3. If $Y \subseteq$ closure, then add all attributes of Z to 'closure'.
4. Stop when a full pass does not add any new attributes.

Example: Computing a Closure

Let's compute $\{A, C\}^+$:

- **Start**: closure $= \{A, C\}$

Given FDs (F):

$A \rightarrow B$
$BC \rightarrow D$
$D \rightarrow E$

Example: Computing a Closure

Let's compute $\{A, C\}^+$:

- **Start**: closure $= \{A, C\}$
- Use $A \rightarrow B$:
  closure $= \{A, B, C\}$

Given FDs (F):

$A \rightarrow B$
$BC \rightarrow D$
$D \rightarrow E$

Example: Computing a Closure

Given FDs (F):

$A \rightarrow B$
$BC \rightarrow D$
$D \rightarrow E$

Let's compute $\{A, C\}^+$:

- **Start**: closure $= \{A, C\}$
- Use $A \rightarrow B$:
  closure $= \{A, B, C\}$
- Use $BC \rightarrow D$:
  (B and C are in closure)
  closure $= \{A, B, C, D\}$

Example: Computing a Closure

Given FDs (F):

$A \rightarrow B$
$BC \rightarrow D$
$D \rightarrow E$

Let's compute $\{A, C\}^+$:

- **Start**: closure $= \{A, C\}$
- Use $A \rightarrow B$:
  closure $= \{A, B, C\}$
- Use $BC \rightarrow D$:
  (B and C are in closure)
  closure $= \{A, B, C, D\}$
- Use $D \rightarrow E$:
  (D is in closure)
  closure $= \{A, B, C, D, E\}$

Example: Computing a Closure

Given FDs (F):

$A \to B$
$BC \to D$
$D \to E$

Let's compute $\{A, C\}^+$:

- **Start**: closure $= \{A, C\}$
- Use $A \to B$:
  closure $= \{A, B, C\}$
- Use $BC \to D$:
  (B and C are in closure)
  closure $= \{A, B, C, D\}$
- Use $D \to E$:
  (D is in closure)
  closure $= \{A, B, C, D, E\}$
- A full scan adds nothing new.
  We are done.

Final Result:
$\{A, C\}^+ = \{A, B, C, D, E\}$

4. The Goal: Using Closures to Find Keys

Why Did We Compute the Closure?

The closure is an essential tool for two critical database design tasks:

1. **Testing if an FD holds**: Does $X \to Y$ follow logically from our set of FDs?
2. **Finding keys for a relation**: What is the minimal set of attributes that can uniquely identify a row?

Algorithm to Test if $X \to Y$ Holds

1. Compute X^+ using the given set of FDs.
2. Check if all attributes of Y are contained in X^+.
3. If $Y \subseteq X^+$, then the FD $X \to Y$ holds.

Example: Using Closure to Test an FD

Given FDs (F):

$A \rightarrow D$
$AB \rightarrow E$
$DE \rightarrow C$

Example: Using Closure to Test an FD

Given FDs (F):

$A \rightarrow D$
$AB \rightarrow E$
$DE \rightarrow C$

Question: Is the FD $AB \rightarrow C$ logically implied by F?

Solution using the closure algorithm:

1. **Compute the closure of the left side,** $\{A, B\}^+$:
   - Start: closure $= \{A, B\}$
   - Use $A \rightarrow D$: closure $= \{A, B, D\}$
   - Use $AB \rightarrow E$: closure $= \{A, B, D, E\}$
   - Use $DE \rightarrow C$: closure $= \{A, B, D, E, C\}$
   - Final closure $\{A, B\}^+ = \{A, B, C, D, E\}$

Example: Using Closure to Test an FD

Given FDs (F):

$A \rightarrow D$
$AB \rightarrow E$
$DE \rightarrow C$

Question: Is the FD $AB \rightarrow C$ logically implied by F?

Solution using the closure algorithm:

1. **Compute the closure of the left side, $\{A, B\}^+$:**
   - Start: closure $= \{A, B\}$
   - Use $A \rightarrow D$: closure $= \{A, B, D\}$
   - Use $AB \rightarrow E$: closure $= \{A, B, D, E\}$
   - Use $DE \rightarrow C$: closure $= \{A, B, D, E, C\}$
   - Final closure $\{A, B\}^+ = \{A, B, C, D, E\}$
2. **Check if the right side, $\{C\}$, is in the closure**:
   - Is $C \in \{A, B, C, D, E\}$? **Yes**.

Example: Using Closure to Test an FD

Given FDs (F):

$A \to D$
$AB \to E$
$DE \to C$

Question: Is the FD $AB \to C$ logically implied by F?

Solution using the closure algorithm:

1. **Compute the closure of the left side, $\{A, B\}^+$:**
   - Start: closure $= \{A, B\}$
   - Use $A \to D$: closure $= \{A, B, D\}$
   - Use $AB \to E$: closure $= \{A, B, D, E\}$
   - Use $DE \to C$: closure $= \{A, B, D, E, C\}$
   - Final closure $\{A, B\}^+ = \{A, B, C, D, E\}$
2. **Check if the right side, $\{C\}$, is in the closure**:
   - Is $C \in \{A, B, C, D, E\}$? **Yes**.
3. **Conclusion**: The FD $AB \to C$ holds.

Formal Definitions: Superkeys and Keys

- A **superkey** for a relation R is a set of attributes K such that K functionally determines *all* attributes of R.
  - Using closures: K is a superkey if K^+ contains all attributes of R.

- A **candidate key** (or just a **key**) is a *minimal* superkey.
  - K is a candidate key if it's a superkey, and removing **any** single attribute from K makes it no longer a superkey.
  - Think of 'minimal' as meaning '**no wasted attributes**'.

Primary Key

The database designer chooses one of the candidate keys to be the **primary key**.

Example: Superkeys vs. Keys

Relation and FDs

MovieStar(<u>title</u>, <u>year</u>, length, genre, studioName, <u>starName</u>)
FD: `title, year` $\rightarrow$ `length, genre, studioName`

Let U = {title, year, length, genre, studioName, starName}

Superkeys

- K = {title, year, starName}
  $K^+ = U$
- K = {title, year, starName, length}
  $K^+ = U$
- K = {title, year, starName, genre}
  $K^+ = U$

The only **Candidate Key** is:
{**title, year, starName**}

Why? Because if we remove any attribute from it, it's no longer a superkey. For example:
$\{title, year\}^+ = \{title, year, length, genre, studioName\} \neq U$.

Algorithm for Finding Candidate Keys

For a relation R with attributes U and FDs F:

1. The goal is to find all *minimal* sets of attributes $K \subseteq U$ such that $K^+ = U$.

A Practical Approach

The brute-force method of checking every subset of U is too slow. A better way:

- **The Key-Finder's Golden Rule:** Any attribute that does not appear on the right-hand side of any FD **must** be in every candidate key.
- Use this as a starting point and compute closures, adding attributes as needed until you find a superkey.
- Once you have a superkey, check if any of its attributes can be removed.

Example of Finding Keys

Relation: Product(name, price, category, color)

FDs:
- `name, category` $\rightarrow$ `price`
- `category` $\rightarrow$ `color`

1. **Analyze attributes**: Notice that 'name' never appears on the RHS of any FD. Therefore, 'name' **must** be part of any candidate key.
2. **Test closures starting with 'name'**:
   - $\{name\}^+ = \{name\}$. Not a superkey.
   - We need to add more attributes. Let's try adding 'category'.
   - $\{name, category\}^+$:
     - Start: $\{name, category\}$
     - Use `name, category` $\rightarrow$ `price`: $\{name, category, price\}$
     - Use `category` $\rightarrow$ `color`: $\{name, category, price, color\}$
     - This is all attributes! So, {**name, category**} **is a superkey**.
3. **Check for minimality**:
   - Is {name, category} minimal? Yes, because its proper subsets $\{name\}$ and $\{category\}$ are not superkeys (we already checked $\{name\}^+$; we can check $\{category\}^+ = \{category, color\}$).

Exercise #2: Find All The Keys

Given relation R(A,B,C,D) and FDs F = $\{AB \rightarrow C,\ C \rightarrow D,\ D \rightarrow A\}$

Find all candidate keys of R.

Exercise #2: Find All The Keys

Given relation R(A,B,C,D) and FDs F = $\{AB \rightarrow C,\ C \rightarrow D,\ D \rightarrow A\}$

Find all candidate keys of R.

Hint

Notice that attribute B never appears on the right-hand side of any FD. What does that tell you about where B must be? Start by computing closures of sets that include B.

Exercise #2: Find All The Keys

Given relation R(A,B,C,D) and FDs F = $\{AB \rightarrow C,\ C \rightarrow D,\ D \rightarrow A\}$

Find all candidate keys of R.

Hint

Notice that attribute B never appears on the right-hand side of any FD. What does that tell you about where B must be? Start by computing closures of sets that include B.

Solution

The candidate keys are: {**A,B**}, {**B,C**}, and {**B,D**}.

- $\{A, B\}^+ = \{A, B, C, D\}$ (Superkey, minimal)
- $\{B, C\}^+ = \{B, C, D, A\}$ (Superkey, minimal)
- $\{B, D\}^+ = \{B, D, A, C\}$ (Superkey, minimal)

Summary

What We've Accomplished

We have now developed a diagnostic toolkit. Given any relational schema, we can:

- Identify its business rules as Functional Dependencies.
- Use the closure algorithm to find all of its keys.
- Definitively diagnose if a design is "bad" by identifying the potential for anomalies.

The Critical Engineering Question

But this leaves us with the next big question: What is the systematic, provably correct **procedure** for *fixing* a bad design? How do we take a problematic relation and decompose it into a set of smaller, well-structured, "good" relations? That procedure is called

Normalization, and our goal is a schema in **Boyce-Codd Normal Form (BCNF)**.

Beyond Functional Dependencies

FDs are the most common, but not the only, type of constraint that governs database design.

1. Multivalued Dependencies (MVDs)

What if dependencies are not one-to-one? Consider an `Employee` with `(EmpID, Skill, Hobby)`. An employee can have multiple skills and multiple hobbies, and their skills and hobbies are completely independent of each other.

- `EmpID` determines a *set* of skills.
- `EmpID` determines a *set* of hobbies.

This relationship, where one attribute determines two or more independent sets of attributes, is a **Multivalued Dependency (MVD)**, written as `EmpID` $\rightarrow\rightarrow$ `Skill` | `Hobby`. MVDs are resolved by decomposing to **4th Normal Form (4NF)**.

2. Minimal (Canonical) Covers

We start with a set of FDs from business rules. But is that set as simple as it can be? It might contain redundant dependencies (e.g., $A \to C$ when $A \to B$ and $B \to C$ already exist). The process of algorithmically reducing a set of FDs to the smallest possible equivalent set is called finding a **Minimal or Canonical Cover**. This is a crucial step in advanced database design.